

Code Export

Source Code PDF Report

Generated 2026-03-16 16:14

Root directory C:\Users\LENOVO\Documents\Java_run\LearningManagementSystem\LearningManagementSystem

Files found 51

Total lines 2,153

Table of Contents

1. Parent: README.md (3 lines)
2. Current: .mvn\wrapper\maven-wrapper.properties (4 lines)
3. Current: pom.xml (82 lines)
4. Current: src\main\java\com\example\LearningManagementSystem\code\CodeToPdf.java (346 lines)
5. Current: src\main\java\com\example\LearningManagementSystem\Controller\CategoryController.java (47 lines)
6. Current: src\main\java\com\example\LearningManagementSystem\Controller\CourseController.java (69 lines)
7. Current: src\main\java\com\example\LearningManagementSystem\Controller\CourseMaterialController.java (50 lines)
8. Current: src\main\java\com\example\LearningManagementSystem\Controller\EnrollmentController.java (54 lines)
9. Current: src\main\java\com\example\LearningManagementSystem\Controller\GlobalExceptionHandler.java (91 lines)
10. Current: src\main\java\com\example\LearningManagementSystem\Controller\InstructorController.java (47 lines)
11. Current: src\main\java\com\example\LearningManagementSystem\Controller\StudentController.java (45 lines)
12. Current: src\main\java\com\example\LearningManagementSystem\Controller\StudentProfileController.java (37 lines)
13. Current: src\main\java\com\example\LearningManagementSystem\DTO\StudentDTO.java (42 lines)
14. Current: src\main\java\com\example\LearningManagementSystem\DTO\StudentProfileDTO.java (70 lines)
15. Current: src\main\java\com\example\LearningManagementSystem\Entity\Category.java (32 lines)
16. Current: src\main\java\com\example\LearningManagementSystem\Entity\Course.java (74 lines)
17. Current: src\main\java\com\example\LearningManagementSystem\Entity\CourseMaterial.java (50 lines)
18. Current: src\main\java\com\example\LearningManagementSystem\Entity\Enrollment.java (52 lines)
19. Current: src\main\java\com\example\LearningManagementSystem\Entity\ErrorResponse.java (41 lines)
20. Current: src\main\java\com\example\LearningManagementSystem\Entity\Instructor.java (84 lines)
21. Current: src\main\java\com\example\LearningManagementSystem\Entity\Student.java (71 lines)
22. Current: src\main\java\com\example\LearningManagementSystem\Entity\StudentProfile.java (66 lines)
23. Current: src\main\java\com\example\LearningManagementSystem\Exception\AlreadyEnrolledException.java (10 lines)
24. Current:
src\main\java\com\example\LearningManagementSystem\Exception\CourseMaterialNotFoundException.java (8 lines)
25. Current: src\main\java\com\example\LearningManagementSystem\Exception\CourseNotFoundException.java (11 lines)
26. Current: src\main\java\com\example\LearningManagementSystem\Exception\DuplicateResourceException.java (8 lines)

27. Current: src\main\java\com\example\LearningManagementSystem\Exception\ProfileAlreadySetException.java (8 lines)

28. Current: src\main\java\com\example\LearningManagementSystem\Exception\ProfileNotFoundException.java (8 lines)

29. Current: src\main\java\com\example\LearningManagementSystem\Exception\ResourceNotFoundException.java (11 lines)

30. Current: src\main\java\com\example\LearningManagementSystem\Exception\StudentNotFoundException.java (8 lines)

31. Current:
src\main\java\com\example\LearningManagementSystem\LearningManagementSystemApplication.java (14 lines)

32. Current: src\main\java\com\example\LearningManagementSystem\Notification\EmailNotificationService.java (13 lines)

33. Current: src\main\java\com\example\LearningManagementSystem\Notification\NotificationService.java (6 lines)

34. Current: src\main\java\com\example\LearningManagementSystem\Notification\SmsNotificationService.java (12 lines)

35. Current: src\main\java\com\example\LearningManagementSystem\Projection\CourseMaterialCount.java (7 lines)

36. Current: src\main\java\com\example\LearningManagementSystem\Repository\CategoryRepo.java (14 lines)

37. Current: src\main\java\com\example\LearningManagementSystem\Repository\CourseMaterialRepo.java (17 lines)

38. Current: src\main\java\com\example\LearningManagementSystem\Repository\CourseRepo.java (18 lines)

39. Current: src\main\java\com\example\LearningManagementSystem\Repository\EnrollmentRepo.java (17 lines)

40. Current: src\main\java\com\example\LearningManagementSystem\Repository\InstructorRepo.java (14 lines)

41. Current: src\main\java\com\example\LearningManagementSystem\Repository\StudentProfileRepo.java (10 lines)

42. Current: src\main\java\com\example\LearningManagementSystem\Repository\StudentRepo.java (8 lines)

43. Current: src\main\java\com\example\LearningManagementSystem\Service\CategoryService.java (50 lines)

44. Current: src\main\java\com\example\LearningManagementSystem\Service\CourseMaterialService.java (52 lines)

45. Current: src\main\java\com\example\LearningManagementSystem\Service\CourseService.java (82 lines)

46. Current: src\main\java\com\example\LearningManagementSystem\Service\EnrollmentService.java (67 lines)

47. Current: src\main\java\com\example\LearningManagementSystem\Service\InstructorService.java (54 lines)

48. Current: src\main\java\com\example\LearningManagementSystem\Service\StudentProfileService.java (51 lines)

49. Current: src\main\java\com\example\LearningManagementSystem\Service\StudentService.java (91 lines)

50. Current: src\main\resources\application.yaml (13 lines)

51. Current:
src\test\java\com\example\LearningManagementSystem\LearningManagementSystemApplicationTests.java (14 lines)

1 Parent: README.md

3 lines

```
1 # all_common_annotations_Project
2 Spring Boot annotations simplify app dev. Core: @SpringBootApplication, @Configuration, @Bean. Stereotypes:
  @Component, @Service, @Repository, @RestController. DI: @Autowired, @Qualifier. Web: @Get...
3
```

2 Current: .mvn\wrapper\maven-wrapper.properties

4 lines

```
1 wrapperVersion=3.3.4
2 distributionType=only-script
3 distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.12/apache-maven-
3.9.12-bin.zip
4
```

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4     <modelVersion>4.0.0</modelVersion>
5     <parent>
6         <groupId>org.springframework.boot</groupId>
7         <artifactId>spring-boot-starter-parent</artifactId>
8         <version>4.0.3</version>
9         <relativePath/> <!-- lookup parent from repository -->
10    </parent>
11    <groupId>com.example</groupId>
12    <artifactId>LearningManagementSystem</artifactId>
13    <version>0.0.1-SNAPSHOT</version>
14    <name>LearningManagementSystem</name>
15    <description>Demo project for Spring Boot</description>
16    <url/>
17    <licenses>
18        <license/>
19    </licenses>
20    <developers>
21        <developer/>
22    </developers>
23    <scm>
24        <connection/>
25        <developerConnection/>
26        <tag/>
27        <url/>
28    </scm>
29    <properties>
30        <java.version>21</java.version>
31    </properties>
32    <dependencies>
33        <dependency>
34            <groupId>org.springframework.boot</groupId>
35            <artifactId>spring-boot-starter-data-jpa</artifactId>
36        </dependency>
37        <dependency>
38            <groupId>org.springframework.boot</groupId>
39            <artifactId>spring-boot-starter-webmvc</artifactId>
40        </dependency>
41
42        <dependency>
43            <groupId>org.springframework.boot</groupId>
44            <artifactId>spring-boot-devtools</artifactId>
45            <scope>runtime</scope>
46            <optional>true</optional>
47        </dependency>
48        <dependency>
49            <groupId>com.mysql</groupId>
50            <artifactId>mysql-connector-j</artifactId>
51            <scope>runtime</scope>
52        </dependency>
53        <!-- iText PDF -->
54        <dependency>
55            <groupId>com.itextpdf</groupId>
56            <artifactId>itext7-core</artifactId>
57            <version>7.2.5</version>
58            <type>pom</type>
59        </dependency>
60        <dependency>
61            <groupId>org.springframework.boot</groupId>
62            <artifactId>spring-boot-starter-data-jpa-test</artifactId>
63            <scope>test</scope>
64        </dependency>
65        <dependency>
66            <groupId>org.springframework.boot</groupId>
67            <artifactId>spring-boot-starter-webmvc-test</artifactId>
68            <scope>test</scope>
69        </dependency>
70    </dependencies>
71
72    <build>

```

```
73     <plugins>
74         <plugin>
75             <groupId>org.springframework.boot</groupId>
76             <artifactId>spring-boot-maven-plugin</artifactId>
77         </plugin>
78     </plugins>
79 </build>
80
81 </project>
82
```

```

1 package com.example.LearningManagementSystem.code;
2 import com.itextpdf.kernel.colors.ColorConstants;
3 import com.itextpdf.kernel.colors.DeviceRgb;
4 import com.itextpdf.kernel.font.PdfFont;
5 import com.itextpdf.kernel.font.PdfFontFactory;
6 import com.itextpdf.kernel.geom.PageSize;
7 import com.itextpdf.kernel.pdf.PdfDocument;
8 import com.itextpdf.kernel.pdf.PdfWriter;
9 import com.itextpdf.layout.Document;
10 import com.itextpdf.layout.element.AreaBreak;
11 import com.itextpdf.layout.element.Cell;
12 import com.itextpdf.layout.element.Paragraph;
13 import com.itextpdf.layout.element.Table;
14 import com.itextpdf.layout.element.Text;
15 import com.itextpdf.layout.properties.AreaBreakType;
16 import com.itextpdf.layout.properties.UnitValue;
17 import com.itextpdf.io.font.constants.StandardFonts;
18
19 import java.io.File;
20 import java.io.IOException;
21 import java.nio.file.*;
22 import java.nio.file.attribute.BasicFileAttributes;
23 import java.time.LocalDateTime;
24 import java.time.format.DateTimeFormatter;
25 import java.util.*;
26
27 /**
28  * CodeToPdf - Extracts all source code files from the current, parent,
29  * and immediate child directories into a formatted PDF document.
30  *
31  * Dependencies (add to your pom.xml or build.gradle):
32  *   com.itextpdf:itext7-core:7.2.5 (or itext7 bundle)
33  *
34  * Compile:
35  *   javac -cp itext7-core-7.2.5.jar:. CodeToPdf.java
36  *
37  * Run:
38  *   java -cp itext7-core-7.2.5.jar:. CodeToPdf
39  *
40  * Or with Maven:
41  *   mvn compile exec:java -Dexec.mainClass=CodeToPdf
42  */
43 public class CodeToPdf {
44
45     // Configuration
46
47     /** Output PDF filename */
48     private static final String OUTPUT_FILE = "code_export.pdf";
49
50     /** Maximum file size to include (skip very large generated files) */
51     private static final long MAX_FILE_SIZE_BYTES = 500_000; // 500 KB
52
53     /** Source code file extensions to include */
54     private static final Set<String> CODE_EXTENSIONS = new LinkedHashSet<>(Arrays.asList(
55         // JVM languages
56         ".java", ".kt", ".groovy", ".scala", ".clj",
57         // Web
58         ".js", ".ts", ".jsx", ".tsx", ".html", ".css", ".scss", ".less",
59         // Python / Ruby / PHP
60         ".py", ".rb", ".php",
61         // Systems / low-level
62         ".c", ".cpp", ".cc", ".h", ".hpp", ".rs", ".go", ".swift",
63         // Data / config
64         ".xml", ".json", ".yaml", ".yml", ".toml", ".properties", ".env",
65         // Shell / scripts
66         ".sh", ".bash", ".zsh", ".bat", ".ps1",
67         // Build
68         ".gradle", ".cmake", "Makefile", "Dockerfile",
69         // Other
70         ".sql", ".md", ".txt"
71     ));

```



```

72
73  /** Directories to skip entirely */
74  private static final Set<String> SKIP_DIRS = new HashSet<>(Arrays.asList(
75      ".git", ".svn", ".hg", "node_modules", "target", "build",
76      "out", ".idea", ".vscode", "__pycache__", ".gradle", "dist", "vendor"
77  ));
78
79  // Colors
80  private static final DeviceRgb COLOR_HEADER_BG    = new DeviceRgb(0x18, 0x5F, 0xA5); // blue
81  private static final DeviceRgb COLOR_HEADER_TEXT  = new DeviceRgb(0xFF, 0xFF, 0xFF);
82  private static final DeviceRgb COLOR_FILE_TITLE   = new DeviceRgb(0x18, 0x5F, 0xA5);
83  private static final DeviceRgb COLOR_CODE_BG      = new DeviceRgb(0xF7, 0xF7, 0xF7);
84  private static final DeviceRgb COLOR_LINE_NUM     = new DeviceRgb(0x99, 0x99, 0x99);
85  private static final DeviceRgb COLOR_TOC_HEADER   = new DeviceRgb(0x18, 0x5F, 0xA5);
86
87  // Font sizes
88  private static final float FONT_TITLE    = 20f;
89  private static final float FONT_SECTION  = 13f;
90  private static final float FONT_FILE     = 11f;
91  private static final float FONT_CODE     = 7.5f;
92  private static final float FONT_TOC      = 10f;
93
94  // State
95  private final List<FileEntry> fileEntries = new ArrayList<>();
96  private int totalLines = 0;
97
98  //
99
100  public static void main(String[] args) throws Exception {
101      new CodeToPdf().run();
102  }
103
104  private void run() throws Exception {
105      Path currentDir = Paths.get("").toAbsolutePath();
106      Path parentDir  = currentDir.getParent();
107
108      System.out.println("=== Code to PDF Extractor ===");
109      System.out.println("Current directory : " + currentDir);
110      System.out.println("Parent  directory : " + (parentDir != null ? parentDir : "(none)"));
111      System.out.println();
112
113      // 1. Collect files
114
115      // Parent directory (top-level files only, not recursive)
116      if (parentDir != null && Files.exists(parentDir)) {
117          collectFiles(parentDir, false, "Parent");
118      }
119
120      // Current directory (recursive – includes all child subdirectories)
121      collectFiles(currentDir, true, "Current");
122
123      System.out.printf("%nFound %d code file(s) with %,d total lines.%n",
124          fileEntries.size(), totalLines);
125
126      if (fileEntries.isEmpty()) {
127          System.out.println("No code files found. Exiting.");
128          return;
129      }
130
131      // 2. Generate PDF
132      generatePdf(currentDir);
133
134      System.out.println("PDF saved to: " + Paths.get(OUTPUT_FILE).toAbsolutePath());
135  }
136
137  // File Collection
138
139  private void collectFiles(Path root, boolean recursive, String scope) throws IOException {
140      if (!Files.exists(root)) return;
141
142      Files.walkFileTree(root, new SimpleFileVisitor<>() {
143          @Override
144          public FileVisitResult preVisitDirectory(Path dir, BasicFileAttributes attrs) {
145              String name = dir.getFileName() == null ? "" : dir.getFileName().toString();
146              // Never recurse into skip-dirs; always allow the root itself
147              if (!dir.equals(root) && SKIP_DIRS.contains(name)) {

```

```

148         return FileVisitResult.SKIP_SUBTREE;
149     }
150     // If not recursive, skip any subdirectory of root
151     if (!recursive && !dir.equals(root)) {
152         return FileVisitResult.SKIP_SUBTREE;
153     }
154     return FileVisitResult.CONTINUE;
155 }
156
157 @Override
158 public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) throws IOException {
159     if (isCodeFile(file) && attrs.size() <= MAX_FILE_SIZE_BYTES) {
160         String content = Files.readString(file);
161         int lines = content.split("\n", -1).length;
162         String relativePath = scope + ": " + root.relativize(file);
163         fileEntries.add(new FileEntry(relativePath, file, content, lines));
164         totalLines += lines;
165         System.out.println("  " + " + relativePath + " (" + lines + " lines)");
166     }
167     return FileVisitResult.CONTINUE;
168 }
169 });
170 }
171
172 private boolean isCodeFile(Path file) {
173     String name = file.getFileName().toString();
174     // Check for exact filename match (e.g. "Makefile", "Dockerfile")
175     if (CODE_EXTENSIONS.contains(name)) return true;
176     // Check extension
177     int dot = name.lastIndexOf('.');
178     if (dot >= 0) {
179         return CODE_EXTENSIONS.contains(name.substring(dot));
180     }
181     return false;
182 }
183
184 // PDF Generation
185
186 private void generatePdf(Path currentDir) throws IOException {
187     PdfWriter writer = new PdfWriter(OUTPUT_FILE);
188     PdfDocument pdfDoc = new PdfDocument(writer);
189     Document doc = new Document(pdfDoc, PageSize.A4);
190     doc.setMargins(40, 40, 40, 40);
191
192     PdfFont sansFont = PdfFontFactory.createFont(StandardFonts.HELVETICA);
193     PdfFont boldFont = PdfFontFactory.createFont(StandardFonts.HELVETICA_BOLD);
194     PdfFont monoFont = PdfFontFactory.createFont(StandardFonts.COURIER);
195
196     // Cover page
197     addCoverPage(doc, sansFont, boldFont, currentDir);
198     doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
199
200     // Table of contents
201     addTableOfContents(doc, sansFont, boldFont);
202     doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
203
204     // Code sections
205     for (int i = 0; i < fileEntries.size(); i++) {
206         FileEntry entry = fileEntries.get(i);
207         addCodeSection(doc, entry, i + 1, sansFont, boldFont, monoFont);
208         if (i < fileEntries.size() - 1) {
209             doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
210         }
211     }
212
213     doc.close();
214 }
215
216 private void addCoverPage(Document doc, PdfFont sansFont, PdfFont boldFont, Path dir)
217     throws IOException {
218     doc.add(new Paragraph("\n\n\n"));
219
220     // Big title
221     doc.add(new Paragraph("Code Export")
222         .setFont(boldFont).setFontSize(FONT_TITLE + 10)

```

```

224         .setFontColor(COLOR_FILE_TITLE)
225         .setMarginBottom(6));
226
227     doc.add(new Paragraph("Source Code PDF Report")
228         .setFont(sansFont).setFontSize(FONT_SECTION)
229         .setFontColor(ColorConstants.DARK_GRAY)
230         .setMarginBottom(24));
231
232     // Stats table
233     Table stats = new Table(UnitValue.createPercentArray(new float[]{40, 60}))
234         .setWidth(UnitValue.createPercentValue(80));
235     addStatRow(stats, "Generated",    LocalDateTime.now()
236         .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")), sansFont, boldFont);
237     addStatRow(stats, "Root directory", dir.toString(), sansFont, boldFont);
238     addStatRow(stats, "Files found",   String.valueOf(fileEntries.size()), sansFont, boldFont);
239     addStatRow(stats, "Total lines",   String.format("%,d", totalLines), sansFont, boldFont);
240     doc.add(stats);
241 }
242
243 private void addStatRow(Table t, String label, String value,
244     PdfFont sans, PdfFont bold) {
245     t.addCell(new Cell().add(new Paragraph(label)
246         .setFont(bold).setFontSize(10)
247         .setFontColor(ColorConstants.DARK_GRAY))
248         .setBorder(null).setPaddingBottom(6));
249     t.addCell(new Cell().add(new Paragraph(value)
250         .setFont(sans).setFontSize(10))
251         .setBorder(null).setPaddingBottom(6));
252 }
253
254 private void addTableOfContents(Document doc, PdfFont sansFont, PdfFont boldFont)
255     throws IOException {
256     doc.add(new Paragraph("Table of Contents")
257         .setFont(boldFont).setFontSize(FONT_SECTION + 2)
258         .setFontColor(COLOR_TOC_HEADER)
259         .setMarginBottom(12));
260
261     for (int i = 0; i < fileEntries.size(); i++) {
262         FileEntry entry = fileEntries.get(i);
263         String label = String.format("%d.  %s  (%,d lines)",
264             i + 1, entry.relativePath, entry.lineCount);
265         doc.add(new Paragraph(label)
266             .setFont(sansFont).setFontSize(FONT_TOC)
267             .setMarginBottom(3));
268     }
269 }
270
271 private void addCodeSection(Document doc, FileEntry entry, int index,
272     PdfFont sansFont, PdfFont boldFont, PdfFont monoFont)
273     throws IOException {
274
275     // Section header bar
276     Table header = new Table(UnitValue.createPercentArray(new float[]{5, 75, 20}))
277         .setWidth(UnitValue.createPercentValue(100))
278         .setMarginBottom(8);
279
280     header.addCell(new Cell().add(new Paragraph(String.valueOf(index))
281         .setFont(boldFont).setFontSize(FONT_FILE)
282         .setFontColor(COLOR_HEADER_TEXT))
283         .setBackgroundColor(COLOR_HEADER_BG)
284         .setBorder(null).setPadding(6));
285
286     header.addCell(new Cell().add(new Paragraph(entry.relativePath)
287         .setFont(boldFont).setFontSize(FONT_FILE)
288         .setFontColor(COLOR_HEADER_TEXT))
289         .setBackgroundColor(COLOR_HEADER_BG)
290         .setBorder(null).setPadding(6));
291
292     header.addCell(new Cell().add(new Paragraph(entry.lineCount + " lines")
293         .setFont(sansFont).setFontSize(FONT_FILE - 1)
294         .setFontColor(COLOR_HEADER_TEXT))
295         .setBackgroundColor(COLOR_HEADER_BG)
296         .setBorder(null).setPadding(6));
297
298     doc.add(header);
299

```

```

300      // Code block with line numbers
301      String[] lines = entry.content.split("\n", -1);
302      int padwidth = String.valueOf(lines.length).length();
303
304      for (int ln = 0; ln < lines.length; ln++) {
305          String lineNum = String.format("%" + padwidth + "d", ln + 1);
306          String codeLine = lines[ln]
307              .replace("\t", " ") // expand tabs
308              .replace("\r", ""); // strip CR
309
310          // Truncate very long lines
311          if (codeLine.length() > 200) {
312              codeLine = codeLine.substring(0, 197) + "...";
313          }
314
315          Text numText = new Text(lineNum + " ").setFont(monoFont)
316              .setFontSize(FONT_CODE).setFontColor(COLOR_LINE_NUM);
317          Text codeText = new Text(codeLine).setFont(monoFont)
318              .setFontSize(FONT_CODE).setFontColor(ColorConstants.BLACK);
319
320          Paragraph p = new Paragraph().add(numText).add(codeText)
321              .setBackgroundColor(ln % 2 == 0 ? COLOR_CODE_BG : ColorConstants.WHITE)
322              .setMargin(0).setPaddingLeft(4).setPaddingRight(4)
323              .setMultipliedLeading(1.3f)
324              .setFixedLeading(FONT_CODE + 2.5f);
325
326          doc.add(p);
327      }
328  }
329
330  // Data class
331
332  private static class FileEntry {
333      final String relativePath;
334      final Path absolutePath;
335      final String content;
336      final int lineCount;
337
338      FileEntry(String relativePath, Path absolutePath, String content, int lineCount) {
339          this.relativePath = relativePath;
340          this.absolutePath = absolutePath;
341          this.content = content;
342          this.lineCount = lineCount;
343      }
344  }
345 }
346

```

```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Entity.Category;
4 import com.example.LearningManagementSystem.Service.CategoryService;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10
11 @RestController
12 @RequestMapping("/api/categories")
13 public class CategoryController {
14
15     private final CategoryService categoryService;
16
17     public CategoryController(CategoryService categoryService) {
18         this.categoryService = categoryService;
19     }
20
21     @GetMapping
22     public ResponseEntity<List<Category>> getAllCategories() {
23         return ResponseEntity.ok(categoryService.getAllCategories());
24     }
25
26     @GetMapping("/{id}")
27     public ResponseEntity<Category> getCategoryById(@PathVariable Long id) {
28         return ResponseEntity.ok(categoryService.getCategoryById(id));
29     }
30
31     @PostMapping
32     public ResponseEntity<Category> createCategory(@RequestBody Category category) {
33         return new ResponseEntity<>(categoryService.createCategory(category), HttpStatus.CREATED);
34     }
35
36     @PutMapping("/{id}")
37     public ResponseEntity<Category> updateCategory(@PathVariable Long id, @RequestBody Category category) {
38         return ResponseEntity.ok(categoryService.updateCategory(id, category));
39     }
40
41     @DeleteMapping("/{id}")
42     public ResponseEntity<Void> deleteCategory(@PathVariable Long id) {
43         categoryService.deleteCategory(id);
44         return ResponseEntity.noContent().build();
45     }
46 }
47
```

```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Entity.Course;
4 import com.example.LearningManagementSystem.Service.CourseService;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10
11 @RestController
12 @RequestMapping("/api/courses")
13 public class CourseController {
14
15     private final CourseService courseService;
16
17     public CourseController(CourseService courseService) {
18         this.courseService = courseService;
19     }
20
21     @GetMapping
22     public ResponseEntity<List<Course>> getAllCourses() {
23         return ResponseEntity.ok(courseService.getAllCourses());
24     }
25
26     @GetMapping("/{id}")
27     public ResponseEntity<Course> getCourseById(@PathVariable Long id) {
28         return ResponseEntity.ok(courseService.getCourseById(id));
29     }
30
31     @GetMapping("/instructor/{instructorId}")
32     public ResponseEntity<List<Course>> getCoursesByInstructor(@PathVariable Long instructorId) {
33         return ResponseEntity.ok(courseService.getCoursesByInstructor(instructorId));
34     }
35
36     @GetMapping("/category/{categoryId}")
37     public ResponseEntity<List<Course>> getCoursesByCategory(@PathVariable Long categoryId) {
38         return ResponseEntity.ok(courseService.getCoursesByCategory(categoryId));
39     }
40
41     @GetMapping("/search")
42     public ResponseEntity<List<Course>> searchCourses(@RequestParam String keyword) {
43         return ResponseEntity.ok(courseService.searchCourses(keyword));
44     }
45
46     @PostMapping
47     public ResponseEntity<Course> createCourse(
48         @RequestBody Course course,
49         @RequestParam Long instructorId,
50         @RequestParam Long categoryId) {
51         return new ResponseEntity<>(courseService.createCourse(course, instructorId, categoryId),
52             HttpStatus.CREATED);
53     }
54
55     @PutMapping("/{id}")
56     public ResponseEntity<Course> updateCourse(
57         @PathVariable Long id,
58         @RequestBody Course course,
59         @RequestParam Long instructorId,
60         @RequestParam Long categoryId) {
61         return ResponseEntity.ok(courseService.updateCourse(id, course, instructorId, categoryId));
62     }
63
64     @DeleteMapping("/{id}")
65     public ResponseEntity<Void> deleteCourse(@PathVariable Long id) {
66         courseService.deleteCourse(id);
67         return ResponseEntity.noContent().build();
68     }
69 }
```

7 Current:

src\main\java\com\example\LearningManagementSystem\Controller\CourseMaterialController.java

```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Entity.CourseMaterial;
4 import com.example.LearningManagementSystem.Service.CourseMaterialService;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10
11 @RestController
12 @RequestMapping("/api/CourseMaterials")
13 public class CourseMaterialController {
14
15     private CourseMaterialService courseMaterialService;
16     public CourseMaterialController(CourseMaterialService courseMaterialService){
17         this.courseMaterialService=courseMaterialService;
18     }
19
20     @GetMapping
21     public List<CourseMaterial> getAllMaterials(){
22         return courseMaterialService.getAllMaterials();
23     }
24
25     @PostMapping
26     public ResponseEntity<CourseMaterial> addMaterial(@RequestBody CourseMaterial material,@RequestParam
String course_name){
27         CourseMaterial addedMaterial= courseMaterialService.addMaterial(material,course_name);
28         return new ResponseEntity<>(addedMaterial,HttpStatus.OK);
29     }
30
31     @GetMapping("/{id}")
32     public ResponseEntity<List<CourseMaterial>> getCourseMaterialById(@PathVariable long id){
33         List<CourseMaterial> courseMaterials=courseMaterialService.getMaterialById(id);
34         return new ResponseEntity<>(courseMaterials,HttpStatus.OK);
35     }
36
37     @DeleteMapping("/{id}")
38     public ResponseEntity<String> deleteCourseMaterial(@PathVariable int id){
39         courseMaterialService.deleteMaterial(id);
40         return ResponseEntity.status(HttpStatus.OK)
41             .body("Course material :"+id+" successfully deleted");
42     }
43
44     @GetMapping("/getCount")
45     public ResponseEntity<List<?>> getCourseMaterialCount(){
46         return courseMaterialService.getMaterialCount();
47     }
48
49 }
50
```

```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Entity.Enrollment;
4 import com.example.LearningManagementSystem.Service.EnrollmentService;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10
11 @RestController
12 @RequestMapping("/api/enrollments")
13 public class EnrollmentController {
14
15     private final EnrollmentService enrollmentService;
16
17     public EnrollmentController(EnrollmentService enrollmentService) {
18         this.enrollmentService = enrollmentService;
19     }
20
21     @GetMapping
22     public ResponseEntity<List<Enrollment>> getAllEnrollments() {
23         return ResponseEntity.ok(enrollmentService.getAllEnrollments());
24     }
25
26     @GetMapping("/{id}")
27     public ResponseEntity<Enrollment> getEnrollmentById(@PathVariable Long id) {
28         return ResponseEntity.ok(enrollmentService.getEnrollmentById(id));
29     }
30
31     @GetMapping("/student/{studentId}")
32     public ResponseEntity<List<Enrollment>> getEnrollmentsByStudent(@PathVariable Long studentId) {
33         return ResponseEntity.ok(enrollmentService.getEnrollmentsByStudent(studentId));
34     }
35
36     @GetMapping("/course/{courseId}")
37     public ResponseEntity<List<Enrollment>> getEnrollmentsByCourse(@PathVariable Long courseId) {
38         return ResponseEntity.ok(enrollmentService.getEnrollmentsByCourse(courseId));
39     }
40
41     @PostMapping
42     public ResponseEntity<Enrollment> enroll(
43         @RequestParam Long studentId,
44         @RequestParam Long courseId) {
45         return new ResponseEntity<>(enrollmentService.enroll(studentId, courseId), HttpStatus.CREATED);
46     }
47
48     @DeleteMapping("/{id}")
49     public ResponseEntity<Void> unenroll(@PathVariable Long id) {
50         enrollmentService.unenroll(id);
51         return ResponseEntity.noContent().build();
52     }
53 }
54
```



```

1 package com.example.LearningManagementSystem.Controller;
2
3
4 import com.example.LearningManagementSystem.Entity.ErrorResponse;
5 import com.example.LearningManagementSystem.Exception.*;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.ExceptionHandler;
9 import org.springframework.web.bind.annotation.RestControllerAdvice;
10 import org.springframework.web.context.request.WebRequest;
11
12 import java.time.LocalDateTime;
13 import java.util.LinkedHashMap;
14 import java.util.Map;
15
16 @RestControllerAdvice
17 public class GlobalExceptionHandler {
18     private Map<String, Object> buildError(HttpStatus status, String message, WebRequest request) {
19         Map<String, Object> body = new LinkedHashMap<>();
20         body.put("timestamp", LocalDateTime.now());
21         body.put("status", status.value());
22         body.put("error", status.getReasonPhrase());
23         body.put("message", message);
24         body.put("path", request.getDescription(false).replace("uri=", ""));
25         return body;
26     }
27     @ExceptionHandler(StudentNotFoundException.class)
28     public ResponseEntity<Map<String, Object>> handelStudentNotFoundException(StudentNotFoundException
exception, WebRequest request){
29         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND,
30             exception.getMessage(), request), HttpStatus.NOT_FOUND);
31     }
32 }
33
34 @ExceptionHandler(ProfileAlreadySetException.class)
35 public ResponseEntity<Map<String, Object>> handelProfileAlreadySetException(ProfileAlreadySetException
exception, WebRequest request){
36     return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST,
37         exception.getMessage(), request), HttpStatus.BAD_REQUEST);
38 }
39 }
40
41 @ExceptionHandler(ProfileNotFoundException.class)
42 public ResponseEntity<Map<String, Object>> handelProfileNotFoundException(ProfileNotFoundException
exception, WebRequest request){
43     return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND,
44         exception.getMessage(), request), HttpStatus.NOT_FOUND);
45 }
46
47 @ExceptionHandler(CourseMaterialNotFoundException.class)
48 public ResponseEntity<Map<String, Object>> handelCourseMaterialNotFoundException(
49     CourseMaterialNotFoundException exception,
50     WebRequest request
51 ){
52     return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND,
53         exception.getMessage(), request), HttpStatus.NOT_FOUND);
54 }
55
56
57 @ExceptionHandler(CourseNotFoundException.class)
58 public ResponseEntity<Map<String, Object>> handelCourseNotFoundException(CourseNotFoundException
exception, WebRequest request){
59     return new
60     ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, exception.getMessage(), request), HttpStatus.NOT_FOUND);
61 }
62
63 @ExceptionHandler(ResourceNotFoundException.class)
64 public ResponseEntity<Map<String, Object>> handleResourceNotFound(
65     ResourceNotFoundException ex, WebRequest request) {
66     return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
67     HttpStatus.NOT_FOUND);

```

```

66     }
67
68     @ExceptionHandler(DuplicateResourceException.class)
69     public ResponseEntity<Map<String, Object>> handleDuplicateResource(
70         DuplicateResourceException ex, WebRequest request) {
71         return new ResponseEntity<>(buildError(HttpStatus.CONFLICT, ex.getMessage(), request),
72             HttpStatus.CONFLICT);
73     }
74
75     @ExceptionHandler(AlreadyEnrolledException.class)
76     public ResponseEntity<Map<String, Object>> handleAlreadyEnrolled(
77         AlreadyEnrolledException ex, WebRequest request) {
78         return new ResponseEntity<>(buildError(HttpStatus.CONFLICT, ex.getMessage(), request),
79             HttpStatus.CONFLICT);
80     }
81
82     @ExceptionHandler(IllegalArgumentException.class)
83     public ResponseEntity<Map<String, Object>> handleIllegalArgument(
84         IllegalArgumentException ex, WebRequest request) {
85         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
86             HttpStatus.BAD_REQUEST);
87     }
88
89     @ExceptionHandler(Exception.class)
90     public ResponseEntity<Map<String, Object>> handleGeneral(
91         Exception ex, WebRequest request) {
92         return new ResponseEntity<>(buildError(HttpStatus.INTERNAL_SERVER_ERROR, "An unexpected error
93             occurred.", request), HttpStatus.INTERNAL_SERVER_ERROR);
94     }
95 }

```

```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Entity.Instructor;
4 import com.example.LearningManagementSystem.Service.InstructorService;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10
11 @RestController
12 @RequestMapping("/api/instructors")
13 public class InstructorController {
14
15     private final InstructorService instructorService;
16
17     public InstructorController(InstructorService instructorService) {
18         this.instructorService = instructorService;
19     }
20
21     @GetMapping
22     public ResponseEntity<List<Instructor>> getAllInstructors() {
23         return ResponseEntity.ok(instructorService.getAllInstructors());
24     }
25
26     @GetMapping("/{id}")
27     public ResponseEntity<Instructor> getInstructorById(@PathVariable Long id) {
28         return ResponseEntity.ok(instructorService.getInstructorById(id));
29     }
30
31     @PostMapping
32     public ResponseEntity<Instructor> createInstructor(@RequestBody Instructor instructor) {
33         return new ResponseEntity<>(instructorService.createInstructor(instructor), HttpStatus.CREATED);
34     }
35
36     @PutMapping("/{id}")
37     public ResponseEntity<Instructor> updateInstructor(@PathVariable Long id, @RequestBody Instructor
instructor) {
38         return ResponseEntity.ok(instructorService.updateInstructor(id, instructor));
39     }
40
41     @DeleteMapping("/{id}")
42     public ResponseEntity<Void> deleteInstructor(@PathVariable Long id) {
43         instructorService.deleteInstructor(id);
44         return ResponseEntity.noContent().build();
45     }
46 }
47
```

```
1 package com.example.LearningManagementSystem.Controller;
2
3
4 import com.example.LearningManagementSystem.DTO.StudentProfileDTO;
5 import com.example.LearningManagementSystem.Entity.Student;
6 import com.example.LearningManagementSystem.Service.StudentService;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.web.bind.annotation.*;
10
11 import java.util.List;
12
13 @RestController
14 @RequestMapping("/api/students")
15 public class StudentController {
16     private StudentService service;
17     public StudentController(StudentService service){
18         this.service=service;
19     }
20
21     @GetMapping
22     public List<StudentProfileDTO> getAllStudents(){
23         return service.getAllStudents();
24     }
25
26     @PostMapping
27     public ResponseEntity<StudentProfileDTO> addStudent(@RequestBody Student student){
28         return ResponseEntity.status(HttpStatus.OK)
29             .body(service.addStudent(student));
30     }
31
32     @DeleteMapping("/{id}")
33     public ResponseEntity<String> deleteStudent(@PathVariable long id){
34         service.deleteStudent(id);
35         return ResponseEntity.status(HttpStatus.OK)
36             .body("deleted student with id:"+id);
37     }
38     @PutMapping("/{id}")
39     public ResponseEntity<String> updateStudent(@PathVariable long id,@RequestBody Student student){
40         service.updateStudent(id,student);
41         return ResponseEntity.status(HttpStatus.OK)
42             .body("updated student with id"+id);
43     }
44 }
45
```

12 Current:

src\main\java\com\example\LearningManagementSystem\Controller\StudentProfileController.java

```
1 package com.example.LearningManagementSystem.Controller;
2
3 import com.example.LearningManagementSystem.Entity.Student;
4 import com.example.LearningManagementSystem.Entity.StudentProfile;
5 import com.example.LearningManagementSystem.Service.StudentProfileService;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11
12 @RestController
13 @RequestMapping("/api/studentprofiles")
14 public class StudentProfileController {
15     private StudentProfileService service;
16     public StudentProfileController(StudentProfileService service){
17         this.service=service;
18     }
19
20     @GetMapping
21     public List<StudentProfile> getAllprofile(){
22         return service.getAllProfile();
23     }
24
25     @PostMapping
26     public StudentProfile addProfile(@RequestBody StudentProfile studentProfile,@RequestParam long id){
27         return service.addProfile(studentProfile,id);
28     }
29
30     @DeleteMapping("/{id}")
31     public ResponseEntity<String> deleteProfile(@PathVariable long id){
32         service.deleteProfile(id);
33         return ResponseEntity.status(HttpStatus.OK)
34             .body("Profile with id:"+id+" deleted");
35     }
36 }
37
```

```
1 package com.example.LearningManagementSystem.DTO;
2
3 public class StudentDTO {
4     private Long studentId;
5     private String name;
6     private String email;
7
8     public String getPhoneNumber() {
9         return phoneNumber;
10    }
11
12    public void setPhoneNumber(String phoneNumber) {
13        this.phoneNumber = phoneNumber;
14    }
15
16    public String getEmail() {
17        return email;
18    }
19
20    public void setEmail(String email) {
21        this.email = email;
22    }
23
24    public String getName() {
25        return name;
26    }
27
28    public void setName(String name) {
29        this.name = name;
30    }
31
32    public Long getStudentId() {
33        return studentId;
34    }
35
36    public void setStudentId(Long studentId) {
37        this.studentId = studentId;
38    }
39
40    private String phoneNumber;
41 }
42
```

```
1 package com.example.LearningManagementSystem.DTO;
2
3 import java.time.LocalDate;
4
5 public class StudentProfileDTO {
6     private Long studentId;
7     private String name;
8     private String email;
9     private String phoneNumber;
10    private String address;
11    private LocalDate dateOfBirth;
12    private String education;
13
14    public Long getStudentId() {
15        return studentId;
16    }
17
18    public void setStudentId(Long studentId) {
19        this.studentId = studentId;
20    }
21
22    public String getName() {
23        return name;
24    }
25
26    public void setName(String name) {
27        this.name = name;
28    }
29
30    public String getEmail() {
31        return email;
32    }
33
34    public void setEmail(String email) {
35        this.email = email;
36    }
37
38    public String getPhoneNumber() {
39        return phoneNumber;
40    }
41
42    public void setPhoneNumber(String phoneNumber) {
43        this.phoneNumber = phoneNumber;
44    }
45
46    public String getAddress() {
47        return address;
48    }
49
50    public void setAddress(String address) {
51        this.address = address;
52    }
53
54    public LocalDate getDateOfBirth() {
55        return dateOfBirth;
56    }
57
58    public void setDateOfBirth(LocalDate dateOfBirth) {
59        this.dateOfBirth = dateOfBirth;
60    }
61
62    public String getEducation() {
63        return education;
64    }
65
66    public void setEducation(String education) {
67        this.education = education;
68    }
69 }
70
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4 import org.jspecify.annotations.NonNull;
5
6 @Entity
7 public class Category {
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long categoryId;
11    @NonNull
12    private String name;
13
14    public Long getCategoryId() {
15        return categoryId;
16    }
17
18    public @NonNull String getName() {
19        return name;
20    }
21
22
23    public void setCategoryId(Long categoryId) {
24        this.categoryId = categoryId;
25    }
26
27    public void setName(@NonNull String name) {
28        this.name = name;
29    }
30
31 }
32
```



```
1 package com.example.LearningManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4 import org.jspecify.annotations.NonNull;
5
6 import java.time.LocalDateTime;
7
8 @Entity
9 public class Course {
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long courseId;
13     @NonNull
14     private String courseName;
15     private String description;
16     private LocalDateTime created_at;
17     @ManyToOne
18     @JoinColumn(name = "instructor_id")
19     private Instructor instructor;
20
21     @ManyToOne
22     @JoinColumn(name = "category_id")
23     private Category category;
24
25     public Category getCategory() {
26         return category;
27     }
28
29     public Long getCourseId() {
30         return courseId;
31     }
32
33     public @NonNull String getCourseName() {
34         return courseName;
35     }
36
37     public String getDescription() {
38         return description;
39     }
40
41     public LocalDateTime getCreated_at() {
42         return created_at;
43     }
44
45     public Instructor getInstructor() {
46         return instructor;
47     }
48
49     public void setCategory(Category category) {
50         this.category = category;
51     }
52
53     public void setCourseName(@NonNull String courseName) {
54         this.courseName = courseName;
55     }
56
57     public void setDescription(String description) {
58         this.description = description;
59     }
60
61     public void setCreated_at(LocalDateTime created_at) {
62         this.created_at = created_at;
63     }
64
65
66     public void setCourseId(Long courseId) {
67         this.courseId = courseId;
68     }
69
70     public void setInstructor(Instructor instructor) {
71         this.instructor = instructor;
```

```
72     }  
73 }  
74
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 public class CourseMaterial {
7
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private long id;
11    private String title;
12    private String file_url;
13
14    @ManyToOne
15    @JoinColumn(name = "course_id")
16    private Course course;
17
18    public long getId() {
19        return id;
20    }
21
22    public void setId(long id) {
23        this.id = id;
24    }
25
26    public String getTitle() {
27        return title;
28    }
29
30    public void setTitle(String title) {
31        this.title = title;
32    }
33
34    public String getFile_url() {
35        return file_url;
36    }
37
38    public void setFile_url(String file_url) {
39        this.file_url = file_url;
40    }
41
42    public Course getCourse() {
43        return course;
44    }
45
46    public void setCourse(Course course) {
47        this.course = course;
48    }
49 }
50
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 import java.time.LocalDateTime;
6
7 @Entity
8 public class Enrollment {
9     @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private Long enrollmentId;
12
13     @ManyToOne
14     @JoinColumn(name = "student_id")
15     private Student student;
16     @ManyToOne
17     @JoinColumn(name = "course_id")
18     private Course course;
19     private LocalDateTime enrolledDate;
20
21     public Long getEnrollmentId() {
22         return enrollmentId;
23     }
24
25     public void setEnrollmentId(Long enrollmentId) {
26         this.enrollmentId = enrollmentId;
27     }
28
29     public LocalDateTime getEnrolledDate() {
30         return enrolledDate;
31     }
32     public void setEnrolledDate(LocalDateTime enrolledDate) {
33         this.enrolledDate = enrolledDate;
34     }
35
36     public Course getCourse() {
37         return course;
38     }
39
40     public void setCourse(Course course) {
41         this.course = course;
42     }
43
44     public Student getStudent() {
45         return student;
46     }
47
48     public void setStudent(Student student) {
49         this.student = student;
50     }
51 }
52
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import java.time.LocalDateTime;
4
5 public class ErrorResponse {
6     private LocalDateTime timestamp;
7     private String error;
8     private String message;
9
10
11     public ErrorResponse(LocalDateTime timestamp, String error, String message) {
12         this.timestamp = timestamp;
13         this.error = error;
14         this.message = message;
15     }
16
17     public LocalDateTime getTimestamp() {
18         return timestamp;
19     }
20
21     public void setTimestamp(LocalDateTime timestamp) {
22         this.timestamp = timestamp;
23     }
24
25     public String getError() {
26         return error;
27     }
28
29     public void setError(String error) {
30         this.error = error;
31     }
32
33     public String getMessage() {
34         return message;
35     }
36
37     public void setMessage(String message) {
38         this.message = message;
39     }
40 }
41
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import com.fasterxml.jackson.annotation.JsonIgnore;
4 import jakarta.persistence.*;
5 import org.jspecify.annotations.NonNull;
6
7 import java.util.List;
8
9 @Entity
10 public class Instructor {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long instructorId;
14     @NonNull
15     private String name;
16     @NonNull
17     private String email;
18     @NonNull
19     private String password;
20     private String specialization;
21     private short experienceYears;
22     @OneToMany(mappedBy = "instructor")
23     @JsonIgnore
24     private List<Course> courses;
25
26     public @NonNull String getName() {
27         return name;
28     }
29
30     public List<Course> getCourses() {
31         return courses;
32     }
33
34     public Long getInstructorId() {
35         return instructorId;
36     }
37
38     public @NonNull String getEmail() {
39         return email;
40     }
41
42     public @NonNull String getPassword() {
43         return password;
44     }
45
46     public String getSpecialization() {
47         return specialization;
48     }
49
50     public short getExperienceYears() {
51         return experienceYears;
52     }
53
54     public void setCourses(List<Course> courses) {
55         this.courses = courses;
56     }
57
58     public void setInstructorId(Long instructorId) {
59         this.instructorId = instructorId;
60     }
61
62     public void setName(@NonNull String name) {
63         this.name = name;
64     }
65
66     public void setEmail(@NonNull String email) {
67         this.email = email;
68     }
69
70     public void setPassword(@NonNull String password) {
71         this.password = password;
```

```
72     }
73
74     public void setSpecialization(String specialization) {
75         this.specialization = specialization;
76     }
77
78     public void setExperienceYears(short experienceYears) {
79         this.experienceYears = experienceYears;
80     }
81
82
83 }
84
```

```
1 package com.example.LearningManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 public class Student {
7     @Id
8     @GeneratedValue(strategy = GenerationType.IDENTITY)
9     private Long studentId;
10
11     private String name;
12
13     @Column(unique = true)
14     private String email;
15
16     private String password;
17
18     private String phoneNumber;
19
20     @OneToOne(mappedBy = "student", cascade = CascadeType.ALL)
21     private StudentProfile profile;
22
23     public Long getStudentId() {
24         return studentId;
25     }
26
27     public void setStudentId(Long studentId) {
28         this.studentId = studentId;
29     }
30
31     public String getEmail() {
32         return email;
33     }
34
35     public void setEmail(String email) {
36         this.email = email;
37     }
38
39     public String getName() {
40         return name;
41     }
42
43     public void setName(String name) {
44         this.name = name;
45     }
46
47     public String getPassword() {
48         return password;
49     }
50
51     public void setPassword(String password) {
52         this.password = password;
53     }
54
55     public String getPhoneNumber() {
56         return phoneNumber;
57     }
58
59     public void setPhoneNumber(String phoneNumber) {
60         this.phoneNumber = phoneNumber;
61     }
62
63     public StudentProfile getProfile() {
64         return profile;
65     }
66
67     public void setProfile(StudentProfile profile) {
68         this.profile = profile;
69     }
70 }
71
```



```
1 package com.example.LearningManagementSystem.Entity;
2
3 import com.fasterxml.jackson.annotation.JsonIgnore;
4 import jakarta.persistence.*;
5
6 import java.time.LocalDate;
7
8 @Entity
9 public class StudentProfile {
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long profileId;
13
14     private String address;
15
16     @Column(name = "date_of_birth")
17     private LocalDate dateOfBirth;
18
19     private String education;
20
21     @OneToOne
22     @JoinColumn(name = "student_id")
23     @JsonIgnore
24     private Student student;
25
26     public Long getProfileId() {
27         return profileId;
28     }
29
30     public void setProfileId(Long profileId) {
31         this.profileId = profileId;
32     }
33
34     public String getEducation() {
35         return education;
36     }
37
38     public void setEducation(String education) {
39         this.education = education;
40     }
41
42     public LocalDate getDateOfBirth() {
43         return dateOfBirth;
44     }
45
46     public void setDateOfBirth(LocalDate dateOfBirth) {
47         this.dateOfBirth = dateOfBirth;
48     }
49
50     public String getAddress() {
51         return address;
52     }
53
54     public void setAddress(String address) {
55         this.address = address;
56     }
57
58     public Student getStudent() {
59         return student;
60     }
61
62     public void setStudent(Student student) {
63         this.student = student;
64     }
65 }
66
```

23 Current:

src\main\java\com\example\LearningManagementSystem\Exception\AlreadyEnrolledException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 import org.springframework.web.bind.annotation.ExceptionHandler;
4
5 public class AlreadyEnrolledException extends RuntimeException {
6     public AlreadyEnrolledException(String message) {
7         super(message);
8     }
9 }
10
```

24 Current:

src\main\java\com\example\LearningManagementSystem\Exception\CourseMaterialNotFoundExcept

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class CourseMaterialNotFoundExcepti extends RuntimeException{
4     public CourseMaterialNotFoundExcepti(String msg) {
5         super(msg);
6     }
7 }
8
```

25 Current:

src\main\java\com\example\LearningManagementSystem\Exception\CourseNotFoundException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class CourseNotFoundException extends RuntimeException{
4     public CourseNotFoundException(String message) {
5         super(message);
6     }
7     public CourseNotFoundException(String resourceName,String name){
8         super(resourceName + " not found with name: " + name);
9     }
10 }
11
```

26 Current:

src\main\java\com\example\LearningManagementSystem\Exception\DuplicateResourceException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class DuplicateResourceException extends RuntimeException {
4     public DuplicateResourceException(String message) {
5         super(message);
6     }
7 }
8
```

27 Current:

src\main\java\com\example\LearningManagementSystem\Exception\ProfileAlreadySetException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class ProfileAlreadySetException extends RuntimeException{
4     public ProfileAlreadySetException(String msg){
5         super(msg);
6     }
7 }
8
```

28 Current:

src\main\java\com\example\LearningManagementSystem\Exception\ProfileNotFoundException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class ProfileNotFoundException extends RuntimeException{
4     public ProfileNotFoundException(String msg){
5         super(msg);
6     }
7 }
8
```

29 Current:

src\main\java\com\example\LearningManagementSystem\Exception\ResourceNotFoundException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class ResourceNotFoundException extends RuntimeException {
4     public ResourceNotFoundException(String message) {
5         super(message);
6     }
7     public ResourceNotFoundException(String resourceName, Long id) {
8         super(resourceName + " not found with id: " + id);
9     }
10 }
11
```


30 Current:

src\main\java\com\example\LearningManagementSystem\Exception\StudentNotFoundException.java

```
1 package com.example.LearningManagementSystem.Exception;
2
3 public class StudentNotFoundException extends RuntimeException{
4     public StudentNotFoundException(String msg){
5         super(msg);
6     }
7 }
8
```

31 Current:

src\main\java\com\example\LearningManagementSystem\LearningManagementSystemApplication.java

```
1 package com.example.LearningManagementSystem;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class LearningManagementSystemApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(LearningManagementSystemApplication.class, args);
11     }
12
13 }
14
```

32 Current:

src\main\java\com\example\LearningManagementSystem\Notification\EmailNotificationService.java

```
1 package com.example.LearningManagementSystem.Notification;
2
3 import org.springframework.stereotype.Service;
4
5 @Service
6 public class EmailNotificationService implements NotificationService{
7
8     @Override
9     public void Notify(String message) {
10         System.out.println("Email sent!!:"+message);
11     }
12 }
13
```

```
1 package com.example.LearningManagementSystem.Notification;
2
3 public interface NotificationService {
4     void Notify(String message);
5 }
6
```

34 Current:

src\main\java\com\example\LearningManagementSystem\Notification\SmsNotificationService.java

```
1 package com.example.LearningManagementSystem.Notification;
2
3 import org.springframework.stereotype.Service;
4
5 @Service
6 public class SmsNotificationService implements NotificationService {
7     @Override
8     public void Notify(String message) {
9         System.out.println("SMS sent!!:" + message);
10    }
11 }
12
```

35 Current:

src\main\java\com\example\LearningManagementSystem\Projection\CourseMaterialCount.java 7 lines

```
1 package com.example.LearningManagementSystem.Projection;
2
3 public interface CourseMaterialCount {
4     String getCourseName();
5     Long getMaterialCount();
6 }
7
```

```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.Category;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.Optional;
8
9 @Repository
10 public interface CategoryRepo extends JpaRepository<Category, Long> {
11     Optional<Category> findByName(String name);
12     boolean existsByName(String name);
13 }
14
```

```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.CourseMaterial;
4 import com.example.LearningManagementSystem.Projection.CourseMaterialCount;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Query;
7 import org.springframework.stereotype.Repository;
8
9 import java.util.List;
10 @Repository
11 public interface CourseMaterialRepo extends JpaRepository<CourseMaterial, Long> {
12
13     List<CourseMaterial> findByCourse_CourseId(long courseId);
14     @Query(value = "select m.course.courseName as courseName, count(m.id) as materialCount CourseMaterial m
group by m.course.courseName", nativeQuery = true)
15     List<CourseMaterialCount> getMaterialCountPerCourse();
16 }
17
```



```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.Course;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.List;
8 import java.util.Optional;
9
10 @Repository
11 public interface CourseRepo extends JpaRepository<Course, Long> {
12     List<Course> findByInstructor_InstructorId(Long instructorId);
13     List<Course> findByCategory_CategoryId(Long categoryId);
14     List<Course> findByCourseNameContainingIgnoreCase(String keyword);
15     boolean existsByCourseName(String courseName);
16     Optional<Course> findByCourseNameIgnoreCase(String courseName);
17 }
18
```

```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.Enrollment;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.List;
8 import java.util.Optional;
9
10 @Repository
11 public interface EnrollmentRepo extends JpaRepository<Enrollment, Long> {
12     List<Enrollment> findByStudent_StudentId(Long studentId);
13     List<Enrollment> findByCourse_CourseId(Long courseId);
14     Optional<Enrollment> findByStudent_StudentIdAndCourse_CourseId(Long studentId, Long courseId);
15     boolean existsByStudent_StudentIdAndCourse_CourseId(Long studentId, Long courseId);
16 }
17
```

```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.Instructor;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.Optional;
8
9 @Repository
10 public interface InstructorRepo extends JpaRepository<Instructor, Long> {
11     Optional<Instructor> findByEmail(String email);
12     boolean existsByEmail(String email);
13 }
14
```

```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.StudentProfile;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 @Repository
8 public interface StudentProfileRepo extends JpaRepository<StudentProfile, Long> {
9 }
10
```

```
1 package com.example.LearningManagementSystem.Repository;
2
3 import com.example.LearningManagementSystem.Entity.Student;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface StudentRepo extends JpaRepository<Student, Long> {
7 }
8
```

```
1 package com.example.LearningManagementSystem.Service;
2
3 import com.example.LearningManagementSystem.Entity.Category;
4 import com.example.LearningManagementSystem.Exception.DuplicateResourceException;
5 import com.example.LearningManagementSystem.Exception.ResourceNotFoundException;
6 import com.example.LearningManagementSystem.Repository.CategoryRepo;
7 import org.springframework.stereotype.Service;
8
9 import java.util.List;
10
11 @Service
12 public class CategoryService {
13
14     private final CategoryRepo categoryRepository;
15
16     public CategoryService(CategoryRepo categoryRepository) {
17         this.categoryRepository = categoryRepository;
18     }
19
20     public List<Category> getAllCategories() {
21         return categoryRepository.findAll();
22     }
23
24     public Category getCategoryById(Long id) {
25         return categoryRepository.findById(id)
26             .orElseThrow(() -> new ResourceNotFoundException("Category", id));
27     }
28
29     public Category createCategory(Category category) {
30         if (categoryRepository.existsByName(category.getName())) {
31             throw new DuplicateResourceException("Category with name '" + category.getName() + "' already
exists.");
32         }
33         return categoryRepository.save(category);
34     }
35
36     public Category updateCategory(Long id, Category updated) {
37         Category existing = getCategoryById(id);
38         if (!existing.getName().equals(updated.getName()) &&
categoryRepository.existsByName(updated.getName())) {
39             throw new DuplicateResourceException("Category with name '" + updated.getName() + "' already
exists.");
40         }
41         existing.setName(updated.getName());
42         return categoryRepository.save(existing);
43     }
44
45     public void deleteCategory(Long id) {
46         getCategoryById(id);
47         categoryRepository.deleteById(id);
48     }
49 }
50
```

```

1  package com.example.LearningManagementSystem.Service;
2
3  import com.example.LearningManagementSystem.Entity.Course;
4  import com.example.LearningManagementSystem.Entity.CourseMaterial;
5  import com.example.LearningManagementSystem.Exception.CourseNotFoundException;
6  import com.example.LearningManagementSystem.Repository.CourseMaterialRepo;
7  import com.example.LearningManagementSystem.Repository.CourseRepo;
8  import org.springframework.http.HttpStatus;
9  import org.springframework.http.ResponseEntity;
10 import org.springframework.stereotype.Service;
11
12 import java.util.List;
13 import java.util.Optional;
14
15 @Service
16 public class CourseMaterialService {
17
18     private CourseMaterialRepo repo;
19     private CourseRepo courseRepo;
20     public CourseMaterialService(CourseMaterialRepo repo, CourseRepo courseRepo) {
21         this.repo = repo;
22         this.courseRepo=courseRepo;
23     }
24
25     public List<CourseMaterial> getAllMaterials(){
26         return repo.findAll();
27     }
28
29     public CourseMaterial addMaterial(CourseMaterial material, String course_name){
30         Course course=courseRepo.findByCourseNameIgnoreCase(course_name)
31             .orElseThrow(()->new CourseNotFoundException("Course",course_name));
32         material.setCourse(course);
33         return repo.save(material);
34     }
35
36     public List<CourseMaterial> getMaterialById(long id){
37         List<CourseMaterial> courseMaterials=repo.findByCourse_CourseId(id);
38         return courseMaterials;
39     }
40
41     public void deleteMaterial(long id){
42         Optional<CourseMaterial> courseMaterial=repo.findById(id);
43         if(courseMaterial.isPresent()){
44             repo.delete(courseMaterial.get());
45         }
46     }
47
48     public ResponseEntity<List<?>> getMaterialCount() {
49         return new ResponseEntity<>(repo.getMaterialCountPerCourse(), HttpStatus.OK);
50     }
51 }
52

```

```

1 package com.example.LearningManagementSystem.Service;
2
3 import com.example.LearningManagementSystem.Entity.Category;
4 import com.example.LearningManagementSystem.Entity.Course;
5 import com.example.LearningManagementSystem.Entity.Instructor;
6 import com.example.LearningManagementSystem.Exception.DuplicateResourceException;
7 import com.example.LearningManagementSystem.Exception.ResourceNotFoundException;
8 import com.example.LearningManagementSystem.Repository.CourseRepo;
9 import org.springframework.stereotype.Service;
10
11 import java.time.LocalDateTime;
12 import java.util.List;
13
14 @Service
15 public class CourseService {
16
17     private final CourseRepo courseRepository;
18     private final InstructorService instructorService;
19     private final CategoryService categoryService;
20
21     public CourseService(CourseRepo courseRepository,
22                         InstructorService instructorService,
23                         CategoryService categoryService) {
24         this.courseRepository = courseRepository;
25         this.instructorService = instructorService;
26         this.categoryService = categoryService;
27     }
28
29     public List<Course> getAllCourses() {
30         return courseRepository.findAll();
31     }
32
33     public Course getCourseById(Long id) {
34         return courseRepository.findById(id)
35             .orElseThrow(() -> new ResourceNotFoundException("Course", id));
36     }
37
38     public List<Course> getCoursesByInstructor(Long instructorId) {
39         return courseRepository.findByInstructor_InstructorId(instructorId);
40     }
41
42     public List<Course> getCoursesByCategory(Long categoryId) {
43         return courseRepository.findByCategory_CategoryId(categoryId);
44     }
45
46     public List<Course> searchCourses(String keyword) {
47         return courseRepository.findByCourseNameContainingIgnoreCase(keyword);
48     }
49
50     public Course createCourse(Course course, Long instructorId, Long categoryId) {
51         if (courseRepository.existsByCourseName(course.getCourseName())) {
52             throw new DuplicateResourceException("Course with name '" + course.getCourseName() + "' already
53             exists.");
54         }
55         Instructor instructor = instructorService.getInstructorById(instructorId);
56         Category category = categoryService.getCategoryById(categoryId);
57         course.setInstructor(instructor);
58         course.setCategory(category);
59         course.setCreated_at(LocalDateTime.now());
60         return courseRepository.save(course);
61     }
62
63     public Course updateCourse(Long id, Course updated, Long instructorId, Long categoryId) {
64         Course existing = getCourseById(id);
65         if (!existing.getCourseName().equals(updated.getCourseName())
66             && courseRepository.existsByCourseName(updated.getCourseName())) {
67             throw new DuplicateResourceException("Course with name '" + updated.getCourseName() + "' already
68             exists.");
69         }
70         Instructor instructor = instructorService.getInstructorById(instructorId);
71         Category category = categoryService.getCategoryById(categoryId);

```



```
70     existing.setCourseName(updated.getCourseName());
71     existing.setDescription(updated.getDescription());
72     existing.setInstructor(instructor);
73     existing.setCategory(category);
74     return courseRepository.save(existing);
75 }
76
77 public void deleteCourse(Long id) {
78     getCourseById(id);
79     courseRepository.deleteById(id);
80 }
81 }
82
```

```
1 package com.example.LearningManagementSystem.Service;
2
3 import com.example.LearningManagementSystem.Entity.Course;
4 import com.example.LearningManagementSystem.Entity.Enrollment;
5 import com.example.LearningManagementSystem.Entity.Student;
6 import com.example.LearningManagementSystem.Exception.AlreadyEnrolledException;
7 import com.example.LearningManagementSystem.Exception.ResourceNotFoundException;
8 import com.example.LearningManagementSystem.Repository.EnrollmentRepo;
9 import com.example.LearningManagementSystem.Repository.StudentRepo;
10 import org.springframework.stereotype.Service;
11
12 import java.time.LocalDateTime;
13 import java.util.List;
14
15 @Service
16 public class EnrollmentService {
17
18     private final EnrollmentRepo enrollmentRepository;
19     private final StudentRepo studentRepository;
20     private final CourseService courseService;
21
22     public EnrollmentService(EnrollmentRepo enrollmentRepository,
23                             StudentRepo studentRepository,
24                             CourseService courseService) {
25         this.enrollmentRepository = enrollmentRepository;
26         this.studentRepository = studentRepository;
27         this.courseService = courseService;
28     }
29
30     public List<Enrollment> getAllEnrollments() {
31         return enrollmentRepository.findAll();
32     }
33
34     public Enrollment getEnrollmentById(Long id) {
35         return enrollmentRepository.findById(id)
36             .orElseThrow(() -> new ResourceNotFoundException("Enrollment", id));
37     }
38
39     public List<Enrollment> getEnrollmentsByStudent(Long studentId) {
40         return enrollmentRepository.findByStudent_StudentId(studentId);
41     }
42
43     public List<Enrollment> getEnrollmentsByCourse(Long courseId) {
44         return enrollmentRepository.findByCourse_CourseId(courseId);
45     }
46
47     public Enrollment enroll(Long studentId, Long courseId) {
48         if (enrollmentRepository.existsByStudent_StudentIdAndCourse_CourseId(studentId, courseId)) {
49             throw new AlreadyEnrolledException("Student " + studentId + " is already enrolled in course " +
50 courseId);
51         }
52         Student student = studentRepository.findById(studentId)
53             .orElseThrow(() -> new ResourceNotFoundException("Student", studentId));
54         Course course = courseService.getCourseById(courseId);
55
56         Enrollment enrollment = new Enrollment();
57         enrollment.setStudent(student);
58         enrollment.setCourse(course);
59         enrollment.setEnrolledDate(LocalDateTime.now());
60         return enrollmentRepository.save(enrollment);
61     }
62
63     public void unenroll(Long enrollmentId) {
64         getEnrollmentById(enrollmentId);
65         enrollmentRepository.deleteById(enrollmentId);
66     }
67 }
```

```
1 package com.example.LearningManagementSystem.Service;
2
3 import com.example.LearningManagementSystem.Entity.Instructor;
4 import com.example.LearningManagementSystem.Exception.DuplicateResourceException;
5 import com.example.LearningManagementSystem.Exception.ResourceNotFoundException;
6 import com.example.LearningManagementSystem.Repository.InstructorRepo;
7 import org.springframework.stereotype.Service;
8
9 import java.util.List;
10
11 @Service
12 public class InstructorService {
13
14     private final InstructorRepo instructorRepository;
15
16     public InstructorService(InstructorRepo instructorRepository) {
17         this.instructorRepository = instructorRepository;
18     }
19
20     public List<Instructor> getAllInstructors() {
21         return instructorRepository.findAll();
22     }
23
24     public Instructor getInstructorById(Long id) {
25         return instructorRepository.findById(id)
26             .orElseThrow(() -> new ResourceNotFoundException("Instructor", id));
27     }
28
29     public Instructor createInstructor(Instructor instructor) {
30         if (instructorRepository.existsByEmail(instructor.getEmail())) {
31             throw new DuplicateResourceException("Instructor with email '" + instructor.getEmail() + "'
already exists.");
32         }
33         return instructorRepository.save(instructor);
34     }
35
36     public Instructor updateInstructor(Long id, Instructor updated) {
37         Instructor existing = getInstructorById(id);
38         if (!existing.getEmail().equals(updated.getEmail()) &&
instructorRepository.existsByEmail(updated.getEmail())) {
39             throw new DuplicateResourceException("Email '" + updated.getEmail() + "' is already in use.");
40         }
41         existing.setName(updated.getName());
42         existing.setEmail(updated.getEmail());
43         existing.setPassword(updated.getPassword());
44         existing.setSpecialization(updated.getSpecialization());
45         existing.setExperienceYears(updated.getExperienceYears());
46         return instructorRepository.save(existing);
47     }
48
49     public void deleteInstructor(Long id) {
50         getInstructorById(id);
51         instructorRepository.deleteById(id);
52     }
53 }
54
```

```
1 package com.example.LearningManagementSystem.Service;
2
3 import com.example.LearningManagementSystem.Entity.Student;
4 import com.example.LearningManagementSystem.Entity.StudentProfile;
5 import com.example.LearningManagementSystem.Exception.ProfileAlreadySetException;
6 import com.example.LearningManagementSystem.Exception.ProfileNotFoundException;
7 import com.example.LearningManagementSystem.Exception.StudentNotFoundException;
8 import com.example.LearningManagementSystem.Repository.StudentProfileRepo;
9 import com.example.LearningManagementSystem.Repository.StudentRepo;
10 import org.springframework.beans.factory.annotation.Autowired;
11 import org.springframework.stereotype.Service;
12
13 import java.util.List;
14
15 @Service
16 public class StudentProfileService {
17     private StudentProfileRepo studentProfileRepo;
18     public StudentProfileService(StudentProfileRepo studentProfileRepo){
19         this.studentProfileRepo=studentProfileRepo;
20     }
21     @Autowired
22     private StudentRepo studentRepo;
23
24     public List<StudentProfile> getAllProfile(){
25         return studentProfileRepo.findAll();
26     }
27
28     public StudentProfile addProfile(StudentProfile studentProfile, long id) {
29         Student student= studentRepo.findById(id)
30             .orElseThrow(()-> new StudentNotFoundException("Student with id:"+id+" is not found"));
31
32         if(student.getProfile()!=null){
33             throw new ProfileAlreadySetException("Student with id:"+id+" has a profile");
34         }
35         student.setProfile(studentProfile);
36         studentProfile.setStudent(student);
37         Student updatedStudent=studentRepo.save(student);
38         return updatedStudent.getProfile();
39     }
40
41     public void deleteProfile(long id) {
42         StudentProfile profile=studentProfileRepo.findById(id)
43             .orElseThrow(()->new ProfileNotFoundException("Profile with id:"+id+" not found"));
44         Student student=profile.getStudent();
45         if(student!=null){
46             student.setProfile(null);
47         }
48         studentProfileRepo.delete(profile);
49     }
50 }
51
```

```
1 package com.example.LearningManagementSystem.Service;
2
3 import com.example.LearningManagementSystem.DTO.StudentProfileDTO;
4 import com.example.LearningManagementSystem.Entity.Student;
5 import com.example.LearningManagementSystem.Entity.StudentProfile;
6 import com.example.LearningManagementSystem.Exception.StudentNotFoundException;
7 import com.example.LearningManagementSystem.Notification.NotificationService;
8 import com.example.LearningManagementSystem.Repository.StudentRepo;
9 import org.springframework.beans.factory.annotation.Qualifier;
10 import org.springframework.stereotype.Service;
11
12 import java.util.ArrayList;
13 import java.util.List;
14
15 @Service
16 public class StudentService {
17     private StudentRepo studentRepo;
18     private NotificationService notificationService;
19     public StudentService(StudentRepo studentRepo,
20         @Qualifier("emailNotificationService") NotificationService notificationService){
21         this.studentRepo=studentRepo;
22         this.notificationService=notificationService;
23     }
24
25     public List<StudentProfileDTO> getAllStudents(){
26         List<Student> students=studentRepo.findAll();
27         List<StudentProfileDTO> studentProfileDTOS=new ArrayList<>();
28         for(Student student:students){
29             studentProfileDTOS.add(convertToStudentDTO(student));
30         }
31         return studentProfileDTOS;
32     }
33     public void updateStudent(long id, Student updateStudent){
34         var student = studentRepo.findById(id)
35             .orElseThrow(() -> new StudentNotFoundException("Student with id:" + id + " not found"));
36
37         // Update student fields
38         if(updateStudent.getName() != null) student.setName(updateStudent.getName());
39         if(updateStudent.getEmail() != null) student.setEmail(updateStudent.getEmail());
40         if(updateStudent.getPassword() != null) student.setPassword(updateStudent.getPassword());
41         if(updateStudent.getPhoneNumber() != null) student.setPhoneNumber(updateStudent.getPhoneNumber());
42
43         // Update profile fields
44         if(updateStudent.getProfile() != null){
45             StudentProfile existingProfile = student.getProfile();
46             if(existingProfile == null){
47                 existingProfile = new StudentProfile(); // create if doesn't exist
48                 existingProfile.setStudent(student);
49                 student.setProfile(existingProfile);
50             }
51             StudentProfile incoming = updateStudent.getProfile();
52             if(incoming.getAddress() != null) existingProfile.setAddress(incoming.getAddress());
53             if(incoming.getDateOfBirth() != null) existingProfile.setDateOfBirth(incoming.getDateOfBirth());
54             if(incoming.getEducation() != null) existingProfile.setEducation(incoming.getEducation());
55         }
56
57         Student saved = studentRepo.save(student);
58         convertToStudentDTO(saved);
59     }
60     private StudentProfileDTO convertToStudentDTO(Student student){
61         StudentProfileDTO studentProfileDTO=new StudentProfileDTO();
62         studentProfileDTO.setStudentId(student.getId());
63         studentProfileDTO.setName(student.getName());
64         studentProfileDTO.setEmail(student.getEmail());
65         studentProfileDTO.setPhoneNumber(student.getPhoneNumber());
66         if(student.getProfile()!=null){
67             studentProfileDTO.setAddress(student.getProfile().getAddress());
68             studentProfileDTO.setDateOfBirth(student.getProfile().getDateOfBirth());
69             studentProfileDTO.setEducation(student.getProfile().getEducation());
70         }
71         return studentProfileDTO;
```

```
72     }
73
74     public StudentProfileDTO addStudent(Student student){
75         StudentProfile studentProfile=student.getProfile();
76         if(studentProfile!=null){
77             notificationService.Notify("Profile successfully created for:"+student.getEmail());
78             studentProfile.setStudent(student);
79             student.setProfile(studentProfile);
80         }
81         Student savedStudent=studentRepo.save(student);
82         return convertToStudentDTO(savedStudent);
83     }
84
85     public void deleteStudent(long id) {
86         Student student=studentRepo.findById(id)
87             .orElseThrow(()->new StudentNotFoundException("Student with id:"+id+" not found"));
88         studentRepo.delete(student);
89     }
90 }
91
```

```
1 spring:
2   application:
3     name: LearningManagementSystem
4
5   datasource:
6     url: jdbc:mysql://localhost:3306/lms_db
7     username: root
8     password: Praneeth@03
9
10  jpa:
11    hibernate:
12      ddl-auto: update
13    show-sql: true
```

51 Current:

src\test\java\com\example\LearningManagementSystem\LearningManagementSystemApplicationTests

```
1 package com.example.LearningManagementSystem;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5
6 @SpringBootTest
7 class LearningManagementSystemApplicationTests {
8
9     @Test
10     void contextLoads() {
11     }
12
13 }
14
```